SQL Query Generator

Natural Language to SQL with IBM Granite 3.3 2B

1. Introduction

Structured Query Language (SQL) is widely used to manage and retrieve data from relational databases. However, writing SQL queries requires knowledge of database structure and SQL syntax, which can be difficult for beginners and non-technical users.

The SQL Query Generator is an intelligent system that converts natural language requests into SQL queries automatically. Users can simply type their query in plain English, and the system generates the corresponding SQL statement.

This system uses Artificial Intelligence and Natural Language Processing (NLP) techniques to analyze the user's request, detect relevant database tables and columns, and generate valid SQL queries. The system also includes security checks to prevent unsafe SQL commands.

The goal of this project is to simplify database querying and make SQL more accessible to users who do not have strong programming or database knowledge.

2. Problem Statement

Many users who interact with databases lack knowledge of SQL syntax and query structure. Writing SQL queries manually can be difficult and time-consuming, especially for beginners.

Incorrect SQL syntax can lead to errors or incorrect results. Therefore, there is a need for a system that can automatically generate SQL queries from natural language input.

The SQL Query Generator solves this problem by allowing users to enter queries in plain English and automatically generating the appropriate SQL statements.

3. Objectives

The main objectives of this project are:

• To convert natural language queries into SQL statements

• To simplify database interaction for beginners

• To reduce errors caused by manual SQL query writing

• To demonstrate the use of AI in SQL query generation

• To provide a simple and user-friendly interface for query generation

4. Tools and Technologies Used

Python

Python is used as the main programming language to implement the project logic.

Google Colab

Google Colab provides a cloud environment to run Python code and execute the model.

Hugging Face Transformers

The Transformers library is used to load and run the AI language model.

IBM Granite 3.3 2B Instruct Model

This model is used to understand natural language queries and assist in SQL generation.

Gradio

Gradio is used to create a web-based user interface where users can enter queries and view results.

SQLite

SQLite is used to define the database schema for generating SQL queries.

ReportLab

ReportLab is used to generate PDF reports of generated SQL queries.

Regular Expressions (Regex)

Regex is used to extract conditions such as numbers and keywords from the user query.

5. System Workflow

The working process of the SQL Query Generator is as follows:

The user enters a request in natural language through the interface.

The system analyzes the input text.

The system detects the relevant database table and columns.

Conditions such as date filters or amount filters are extracted.

The system generates a valid SQL query.

The SQL query is displayed to the user.

6. Implementation

The system is implemented using Python in Google Colab.

First, the required libraries such as Transformers, Torch, and Gradio are installed. The IBM Granite 3.3 2B Instruct model is then loaded using the Hugging Face Transformers library.

A database schema containing four tables is defined:

• Users

• Orders

• Products

• Transactions

The system analyzes the natural language input and identifies keywords related to table names and column names.

A function is used to generate SQL queries based on the detected information. The system also includes a security check to block dangerous SQL commands such as DROP, DELETE, and ALTER.

A Gradio interface is created to allow users to interact with the system easily.

7. Database Schema

Table: users

• id – INTEGER PRIMARY KEY

• name – TEXT

• email – TEXT

• signup_date – DATE

• age – INTEGER

• country – TEXT

• status – TEXT

Table: orders

• id – INTEGER PRIMARY KEY

• user_id – INTEGER

• product_name – TEXT

• amount – REAL

• order_date – DATE

• status – TEXT

Table: products

• id – INTEGER PRIMARY KEY

• name – TEXT

• price – REAL

• category – TEXT

• stock – INTEGER

Table: transactions

• id – INTEGER PRIMARY KEY

• user_id – INTEGER

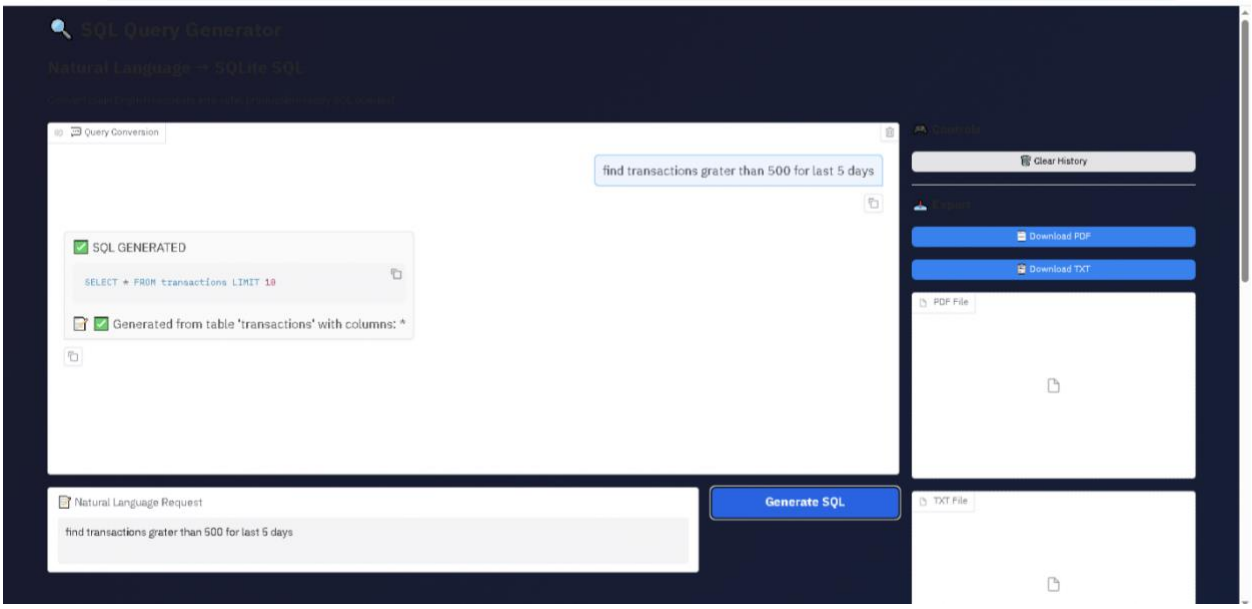
• amount – REAL

• date – DATE

• type – TEXT

8. Output Examples

Example 1



Input Query

Find transactions greater than 500 for last 5 days

Generated SQL

SELECT * FROM transactions

WHERE amount > 500

AND DATE(date) >= DATE('now','-5 days')

LIMIT 10;

Explanation

The system detects:

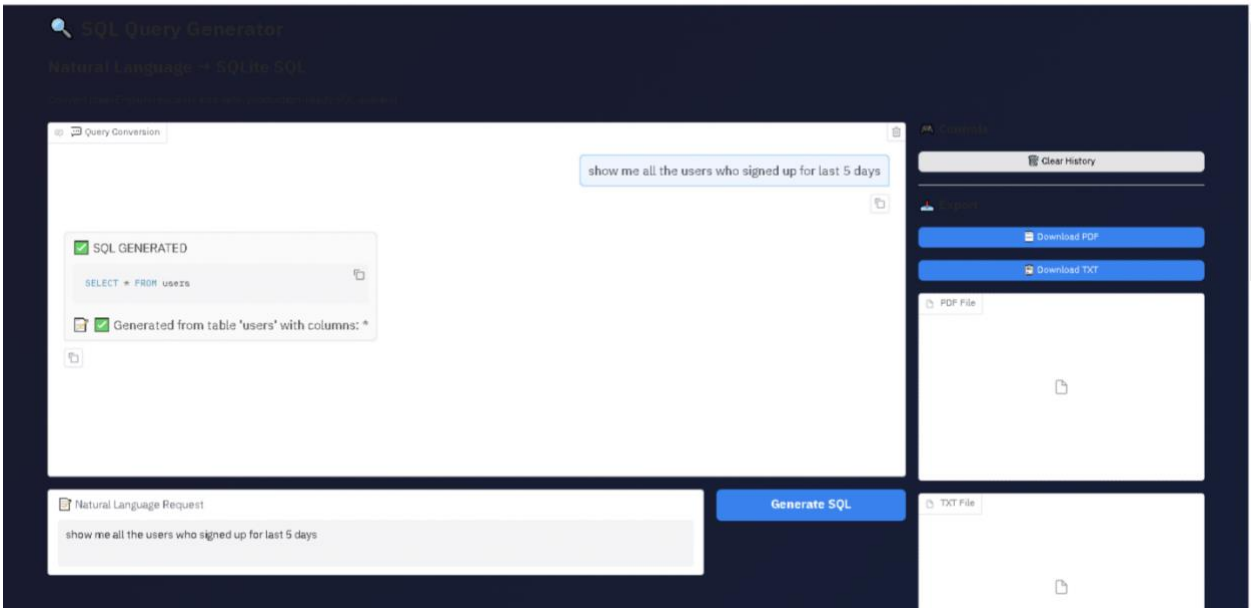
Table: transactions

Condition 1: amount greater than 500

Condition 2: transactions from the last 5 days

The SQL query combines both conditions using the AND operator.

Example 2



Input Query

Show me all the users who signed up for last 5 days

Generated SQL

SELECT * FROM users

WHERE DATE(signup_date) >= DATE('now','-5 days')

LIMIT 10;

Explanation

The system identifies:

Table: users

Column: signup_date

Condition: records from the last 5 days

It then generates the SQL query with the appropriate date filter.

9. Advantages

• Makes SQL query generation easier

• Reduces manual query writing

• Helps beginners understand SQL queries

• Saves time while creating queries

• Demonstrates practical use of Artificial Intelligence

10. Conclusion

The SQL Query Generator successfully demonstrates how artificial intelligence can simplify database querying. By converting natural language queries into SQL commands, the system allows users to interact with databases without requiring detailed knowledge of SQL syntax.

The project highlights the potential of AI-based tools in improving accessibility and efficiency in database management systems. Future improvements may include support for more complex SQL operations such as joins, grouping, and integration with real databases.

11.Source Code

```
!pip install -U transformers


# Use a pipeline as a high-level helper

from transformers import pipeline


pipe = pipeline("text-generation", model="ibm-granite/granite-3.3-2b-instruct")

messages = [

    {"role": "user", "content": "Who are you?"},

]

pipe(messages)


# Load model directly

from transformers import AutoTokenizer, AutoModelForCausalLM


tokenizer = AutoTokenizer.from_pretrained("ibm-granite/granite-3.3-2b-instruct")

model = AutoModelForCausalLM.from_pretrained("ibm-granite/granite-3.3-2b-instruct")
```

```
messages = [
    {"role": "user", "content": "Who are you?"},
]
inputs = tokenizer.apply_chat_template(
       messages,
       add_generation_prompt=True,
       tokenize=True,
       return_dict=True,
       return_tensors="pt",
).to(model.device)

outputs = model.generate(**inputs, max_new_tokens=40)
print(tokenizer.decode(outputs[0][inputs["input_ids"].shape[-1]:]))

!pip install -q gradio transformers torch black autopep8 reportlab requests

!pip install -q gradio transformers torch black autopep8 reportlab requests sqlite3

import gradio as gr
from datetime import datetime
from pathlib import Path
import json
import sqlite3
import re
from reportlab.lib.pagesizes import letter
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, PageBreak,
Table, TableStyle
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.lib.units import inch
from reportlab.lib import colors
```

```
import requests
import urllib.parse

# ==============================================================================
# DATABASE SCHEMA
# ==============================================================================
SCHEMA = {
    'users': {
        'id': 'INTEGER PRIMARY KEY',
        'name': 'TEXT',
        'email': 'TEXT',
        'signup_date': 'DATE',
        'age': 'INTEGER',
        'country': 'TEXT',
        'status': 'TEXT'
    },
    'orders': {
        'id': 'INTEGER PRIMARY KEY',
        'user_id': 'INTEGER',
        'product_name': 'TEXT',
        'amount': 'REAL',
        'order_date': 'DATE',
        'status': 'TEXT'
    },
    'products': {
        'id': 'INTEGER PRIMARY KEY',
        'name': 'TEXT',
        'price': 'REAL',
        'category': 'TEXT',
        'stock': 'INTEGER'
```

```
    },
    'transactions': {
        'id': 'INTEGER PRIMARY KEY',
        'user_id': 'INTEGER',
        'amount': 'REAL',
        'date': 'DATE',
        'type': 'TEXT'
    }
}

# ==============================================================================
# SECURITY & SQL GENERATION
# ==============================================================================
DANGEROUS_KEYWORDS = ['DROP', 'DELETE', 'ALTER', 'TRUNCATE', 'INSERT', 'UPDATE',
'EXEC', 'EXECUTE']
DANGEROUS_CHARS = [';', '--', '/*', '*/', 'xp_', 'sp_']


def is_safe(text):
    """Check if text contains dangerous SQL keywords"""
    text_upper = text.upper()
    for keyword in DANGEROUS_KEYWORDS:
        if keyword in text_upper:
            return False
    for char in DANGEROUS_CHARS:
        if char in text:
            return False
    return True


def get_table_name(query_text):
    """Intelligently infer table name from natural language"""
```

```
    query_lower = query_text.lower()

    keyword_map = {
        'users': ['user', 'users', 'customer', 'customers', 'account', 'accounts',
'profile'],
        'orders': ['order', 'orders', 'purchase', 'purchases', 'buy', 'bought'],
        'products': ['product', 'products', 'item', 'items', 'goods'],
        'transactions': ['transaction', 'transactions', 'payment', 'payments',
'transfer']
    }

    for table, keywords in keyword_map.items():
        for keyword in keywords:
            if keyword in query_lower:
                return table

    return 'users'  # Default table

def get_column_names(table_name, query_text):
    """Map natural language to actual column names"""
    query_lower = query_text.lower()
    columns = list(SCHEMA[table_name].keys())
    selected_cols = []

    column_map = {
        'name': ['name', 'username', 'full name'],
        'email': ['email', 'mail', 'address'],
        'signup_date': ['signup', 'created', 'joined', 'registered', 'date',
'month'],
        'age': ['age', 'years'],
        'amount': ['amount', 'price', 'cost', 'total'],
```

```
        'date': ['date', 'when'],
        'status': ['status', 'state'],
        'country': ['country', 'location', 'place']
    }

    for col, keywords in column_map.items():
        if col in columns:
            for keyword in keywords:
                if keyword in query_lower:
                    selected_cols.append(col)
                    break

    if not selected_cols:
        selected_cols = ['*']
    else:
        selected_cols = list(dict.fromkeys(selected_cols))  # Remove duplicates

    return selected_cols

def extract_conditions(query_text):
    """Extract WHERE conditions from natural language"""
    query_lower = query_text.lower()
    conditions = []

    # Last month
    if 'last month' in query_lower:
        conditions.append("DATE(signup_date) >= DATE('now', '-1 month')")

    # Last year
    if 'last year' in query_lower or 'this year' in query_lower:
```

```
        conditions.append("DATE(signup_date) >= DATE('now', '-1 year')")

    # Specific date patterns
    if 'last week' in query_lower:
        conditions.append("DATE(signup_date) >= DATE('now', '-7 days')")

    if 'today' in query_lower:
        conditions.append("DATE(signup_date) = DATE('now')")

    # Active/inactive
    if 'active' in query_lower:
        conditions.append("status = 'active'")
    elif 'inactive' in query_lower:
        conditions.append("status = 'inactive'")

    # Amount filtering
    amount_match = re.search(r'more than|greater than|above\s+([0-9]+)', query_lower)
    if amount_match:
        conditions.append(f"amount > {amount_match.group(1)}")

    amount_match = re.search(r'less than|below\s+([0-9]+)', query_lower)
    if amount_match:
        conditions.append(f"amount < {amount_match.group(1)}")

    return conditions

def generate_sql(query_text):
    """Generate SQLite SQL from natural language"""

    if not is_safe(query_text):
```

```
        return None, "❌ UNSAFE: Query contains dangerous SQL keywords!"

    table = get_table_name(query_text)
    columns = get_column_names(table, query_text)
    columns_str = ', '.join(columns) if columns != ['*'] else '*'

    # Build base query
    sql = f"SELECT {columns_str} FROM {table}"

    # Add conditions
    conditions = extract_conditions(query_text)
    if conditions:
        sql += " WHERE " + " AND ".join(conditions)

    # Add limit
    if 'all' not in query_text.lower():
        sql += " LIMIT 10"

    return sql, f"✅ Generated from table '{table}' with columns: {columns_str}"

query_history = []

def process_query(user_query, chat_history):
    """Process database query request"""

    if not user_query.strip():
        return chat_history

    chat_history.append((user_query, "Processing..."))
```

```
    sql_query, explanation = generate_sql(user_query)

    if sql_query is None:
        response = f"❌ ERROR\n\n{explanation}"
    else:
        response = f"✅ SQL GENERATED\n\n```sql\n{sql_query}\n```\n\n📝
{explanation}"
        query_history.append({
            'user': user_query,
            'sql': sql_query,
            'timestamp': datetime.now().isoformat()
        })

    chat_history[-1] = (user_query, response)
    return chat_history

def clear_chat():
    global query_history
    query_history = []
    return []

def download_pdf():
    if not query_history:
        return "❌ No queries to export!", None

    timestamp = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
    filename = f"sql_queries_{timestamp}.pdf"

    try:
        doc = SimpleDocTemplate(filename, pagesize=letter)
```

```
        styles = getSampleStyleSheet()
        story = []

        title_style = ParagraphStyle(
            'CustomTitle',
            parent=styles['Heading1'],
            fontSize=26,
            textColor='#FF6B6B',
            spaceAfter=30,
            alignment=1
        )

        story.append(Paragraph("🔍 SQL Query Report", title_style))
        story.append(Paragraph(f"Generated: {datetime.now()}", styles['Normal']))
        story.append(Spacer(1, 0.3*inch))

        for i, query in enumerate(query_history, 1):
            story.append(Paragraph(f"<b>Query #{i}</b>", styles['Heading2']))
            story.append(Paragraph(f"<b>User Request:</b> {query['user']}",
styles['Normal']))
            story.append(Spacer(1, 0.1*inch))
            story.append(Paragraph(f"<b>Generated SQL:</b>", styles['Normal']))
            story.append(Paragraph(f"<font
color='#4ECDC4'><code>{query['sql']}</code></font>", styles['Normal']))
            story.append(Spacer(1, 0.2*inch))

        doc.build(story)
        return f"✅ PDF Downloaded: {filename}", filename
    except Exception as e:
        return f"❌ Error: {str(e)}", None
```

```
def download_txt():
    if not query_history:
        return "❌ No queries to export!", None

    content = f"SQL Query History Report\nGenerated: {datetime.now()}\n{'='*60}\n\n"

    for i, query in enumerate(query_history, 1):
        content += f"Query #{i}\n"
        content += f"User Request: {query['user']}\n"
        content += f"Generated SQL: {query['sql']}\n"
        content += f"Timestamp: {query['timestamp']}\n"
        content += f"{'-'*60}\n\n"

    timestamp = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
    filename = f"sql_queries_{timestamp}.txt"

    try:
        Path(filename).write_text(content)
        return f"✅ TXT Downloaded: {filename}", filename
    except Exception as e:
        return f"❌ Error: {str(e)}", None

def generate_whatsapp_link():
    if not query_history:
        return "❌ No queries to share!"

    message = "🔍 *SQL Queries Generated*\n\n"
    for i, query in enumerate(query_history[-5:], 1):
        message += f"*Query {i}:* {query['user']}\n"
```

```
        message += f"`{query['sql']}`\n\n"

    encoded_message = urllib.parse.quote(message)
    whatsapp_link = f"https://wa.me/?text={encoded_message}"

    return f"✅ WhatsApp Link Created!\n\n🔗 {whatsapp_link}\n\n📱 Click to open
WhatsApp and share!"

def generate_facebook_link():
    if not query_history:
        return "❌ No queries to share!"

    message = "Check out these SQL queries I generated: "
    for query in query_history[-3:]:
        message += f"{query['user']} → {query['sql']} | "

    encoded_message = urllib.parse.quote(message)
    facebook_link =
f"https://www.facebook.com/sharer/sharer.php?quote={encoded_message}"

    return f"✅ Facebook Link Created!\n\n🔗 {facebook_link}\n\n📱 Click to share on
Facebook!"

print("🚀 Loading SQL Query Generator...")

with gr.Blocks(
    title="SQL Query Generator",
    theme=gr.themes.Base(),
    css="""
    body { background: linear-gradient(135deg, #1a1a2e 0%, #16213e 100%); }
```

```
    .gradio-container { background: linear-gradient(135deg, #1a1a2e 0%, #16213e
100%); }

    .chatbot-wrapper { background: #0f3460; border-radius: 15px; }

    .button-primary { background: linear-gradient(135deg, #FF6B6B 0%, #FF8E72 100%);
}

    .button-secondary { background: linear-gradient(135deg, #4ECDC4 0%, #44A08D
100%); }

    .button-danger { background: linear-gradient(135deg, #FF6B6B 0%, #EE5A6F 100%); }

    """

) as demo:


    gr.Markdown("""

    # 🔍 SQL Query Generator

    ## Natural Language → SQLite SQL


    Convert plain English requests into safe, production-ready SQL queries!

    """)


    with gr.Row():

        with gr.Column(scale=3):

            chatbot = gr.Chatbot(

                label="💬 Query Conversion",

                height=450,

                show_copy_button=True,

                bubble_full_width=False

            )


            with gr.Row():

                query_input = gr.Textbox(

                    label="📝 Natural Language Request",

                    placeholder="e.g., Show me all users who signed up last month",
```

```
                lines=2,
                scale=4
            )
            send_btn = gr.Button("Generate SQL", scale=1, size="lg",
variant="primary")

        send_btn.click(process_query, inputs=[query_input, chatbot],
outputs=[chatbot])
        query_input.submit(process_query, inputs=[query_input, chatbot],
outputs=[chatbot])

    with gr.Column(scale=1):
        gr.Markdown("""
        ### 🎮 Controls
        """)

        clear_btn = gr.Button("🗑️ Clear History", variant="stop", size="sm")
        clear_btn.click(clear_chat, outputs=[chatbot])

        gr.Markdown("---")
        gr.Markdown("### 📥 Export")

        pdf_btn = gr.Button("📄 Download PDF", size="sm", variant="primary")
        txt_btn = gr.Button("📋 Download TXT", size="sm", variant="primary")

        pdf_file = gr.File(label="PDF File", visible=True)
        txt_file = gr.File(label="TXT File", visible=True)

        export_status = gr.Textbox(label="Status", interactive=False, lines=2)

        pdf_btn.click(download_pdf, outputs=[export_status, pdf_file])
```

```
txt_btn.click(download_txt, outputs=[export_status, txt_file])

gr.Markdown("---")
gr.Markdown("### 📱 Share")

whatsapp_btn = gr.Button("💬 WhatsApp", size="sm", variant="secondary")
facebook_btn = gr.Button("👍 Facebook", size="sm", variant="secondary")

share_output = gr.Textbox(label="Share Link", interactive=False, lines=5)

whatsapp_btn.click(generate_whatsapp_link, outputs=[share_output])
facebook_btn.click(generate_facebook_link, outputs=[share_output])

gr.Markdown("---")
gr.Markdown("""
### 📊 Schema Info

**Tables:**
- users
- orders
- products
- transactions

**Supported Requests:**
- User queries (last month, etc)
- Order lookups
- Product searches
- Transaction reports
""")
```

```
print("\n✅ SQL Query Generator Ready!\n")

demo.launch(share=True, show_error=True, show_api=False)
```